Modelling Human Activity States Using Hidden Markov Models

Jolly Gift Burabyo · Formative 2 – Hidden Markov Models · 29 October 2026 · Code & data: https://github.com/Burabyo/hmm-activity-recognition

1. Background and motivation

Students often spend long, unbroken hours studying, which is linked to poor posture, fatigue and reduced concentration. My use case is a **phone-based activity monitor for students** that detects prolonged inactivity (the phone lying *still* on a desk, or the student *standing* idle) and nudges them to move, while recognising real activity (*walking*, *jumping*/exercise) so it does not interrupt genuine breaks. The true activity is never observed directly; only noisy accelerometer and gyroscope readings are. A Hidden Markov Model fits this problem: it treats the activity as a hidden state that evolves over time and links it probabilistically to the sensor features, so it can infer a temporally consistent activity timeline from the sensor stream.

2. Data collection and pre-processing

Participant / device	Jolly Gift Burabyo · iPhone 11 Pro (iOS) · Sensor Logger v1.68
Sensors	Accelerometer (x, y, z), Gyroscope (x, y, z), plus Gravity (used to rebuild the raw accelerometer)
Sampling rate	Set to **100 Hz**; measured from timestamps: **99.3 Hz** (Δt = 10.07 ms) on every file
Training data	**52 files**: 13 clips × 4 activities, ~10 s each (bedroom). After cleaning: still 111 s, standing 110 s, walking 114 s, jumping 117 s (all > 1 min 30 s)
Unseen test data	2 continuous recordings (41 s and 44 s) made later in a **different room (living room)**, each containing several activities in sequence

Videos of data collection (full body, face visible, phone recording in hand):

Activity	Video link
Standing	https://drive.google.com/file/d/1yA_DP3qB3pFlJeSCUPvisG_z1yE_k0aB/view?usp=sharing
Walking	https://drive.google.com/file/d/1Vyu1zIJ2cXOFrPhvDf8uZc9PPhVjlmmy/view?usp=sharing
Jumping	https://drive.google.com/file/d/1cam80EsMgi6tWBkMEZ8WwPjal0GsLBZj/view?usp=sharing
Still	https://drive.google.com/file/d/12MuhzI1npKDIN6ipxp3qi3YnDbDqRG-c/view?usp=sharing

Pre-processing

- **Raw accelerometer:** Sensor Logger's *Accelerometer* stream is linear acceleration (gravity removed), so the *Gravity* stream was added back. This keeps phone orientation (flat on a table vs. held upright), which helps separate *still* from *standing*.
- **Sampling-rate harmonisation:** iOS delivered 99.3 Hz rather than the nominal 100 Hz. All streams were aligned on `seconds_elapsed` and linearly interpolated onto an exact **100 Hz grid**. Any phone (e.g. a 50 Hz or 200 Hz Android) can be resampled onto the same grid before windowing, so windows always contain the same number of samples.
- **Cleaning:** the first and last 1 s of each training clip were trimmed to remove button presses. In the test files, moments of handling the phone (picking it up or putting it down) are labelled *transition*: they are decoded but not scored.
- **Windowing: 2 s windows (200 samples) with 50 % overlap (1 s hop)**. Walking and jumping run at about 2 Hz, so each window holds about 4 full motion cycles. The FFT resolution is 100/200 = 0.5 Hz (zero-padded to 1024 points for the peak estimate). A 1 s hop gives one HMM observation per second. This produced 373 training windows (91–97 per activity).

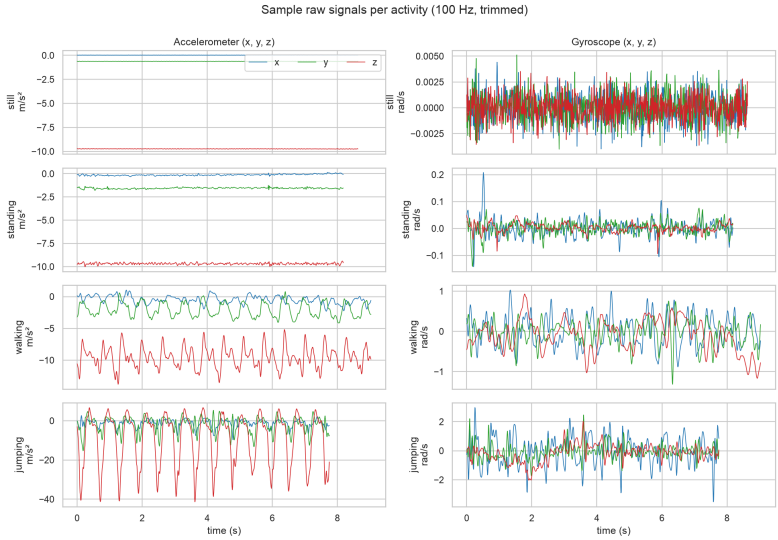


Figure 1 – One clip per activity (raw accelerometer and gyroscope after resampling to 100 Hz). Amplitude and rhythm differ clearly between activities.

3. HMM setup and implementation details

Features (8 per window), computed on the magnitudes a=‖acc‖ and ω=‖gyro‖:

Time domain	**acc_std** (motion intensity) · **acc_sma** (signal magnitude area, total dynamic acceleration) · **gyro_mean** (rotation: zero on the table, small tremor when hand-held) · **tilt** (phone orientation relative to gravity) · **corr_yz** (axis coupling during vertical bouncing)
Frequency domain (FFT)	**dom_freq** (step/jump rhythm ~1.6–2 Hz) · **spec_energy** (0.5–20 Hz power) · **spec_entropy** (peaked spectrum for rhythmic motion vs. flat spectrum for noise)
Normalisation	The intensity features span several orders of magnitude, so they were $\log_{10}$-transformed first to make them roughly Gaussian. All features were then **Z-scored** using the training-set mean and standard deviation only (re-used on the test set, so no leakage). This puts every feature on the same scale for the Gaussian emissions.

Hidden states Z	still, standing, walking, jumping (K = 4), one state per 1-s step
Observations X	the 8-dimensional Z-scored feature vector of each window
Transitions A	4×4 matrix, $A_{ij} = P(z_{t+1}=j \mid z_t=i)$, learned by Baum–Welch
Emissions B	one diagonal-covariance Gaussian per state, $\mathcal{N}(x; \mu_k, \sigma^2_k)$ (continuous observations)
Initial π	uniform: I make no assumption about the activity at the start of a recording

Algorithms. Baum–Welch (EM) learns A, μ and σ² from the data. Its E-step is a *scaled* forward–backward pass, which avoids numerical underflow. It stops when the log-likelihood gain per observation drops below ε = 10^{-4}. **Viterbi** (computed in log space) then decodes the single most likely activity sequence. This suits my evaluation metrics: sensitivity, specificity and accuracy all need one hard label per window, and the transition model discourages isolated label flips. Everything is implemented from scratch in NumPy. The implementation was first validated on synthetic data: the log-likelihood increased monotonically, A was recovered to within 0.04, and the Viterbi path was 99.5 % correct.

Training. Each training clip contains only one activity, so on its own it holds no information about transitions. The clips' window sequences were therefore chained into pseudo-continuous sessions that follow a plausible routine (still → standing → walking → jumping → walking → standing). The remaining clips were added as separate sequences. A pseudo-count of 0.1 keeps every transition possible. The emissions were initialised from the labelled windows, and Baum–Welch converged in **2 iterations** because the classes are well separated. As a robustness check, I also ran Baum–Welch **without labels**, starting from k-means clusters. It converged in 3 iterations, rediscovered all four activities (100 % agreement on the training data) and reached 86.3 % on the test data.

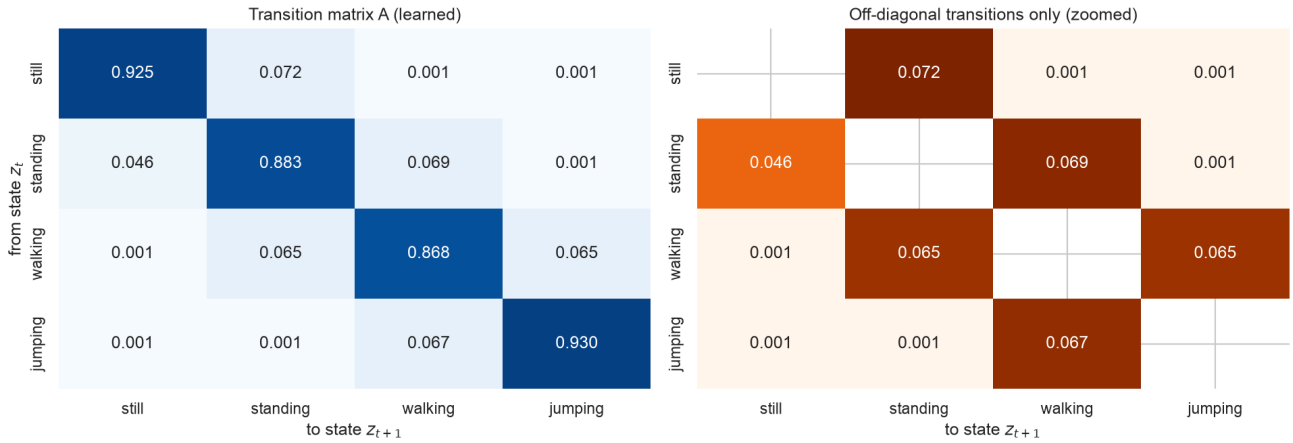


Figure 2 – Learned transition matrix (left) and its off-diagonal entries (right).

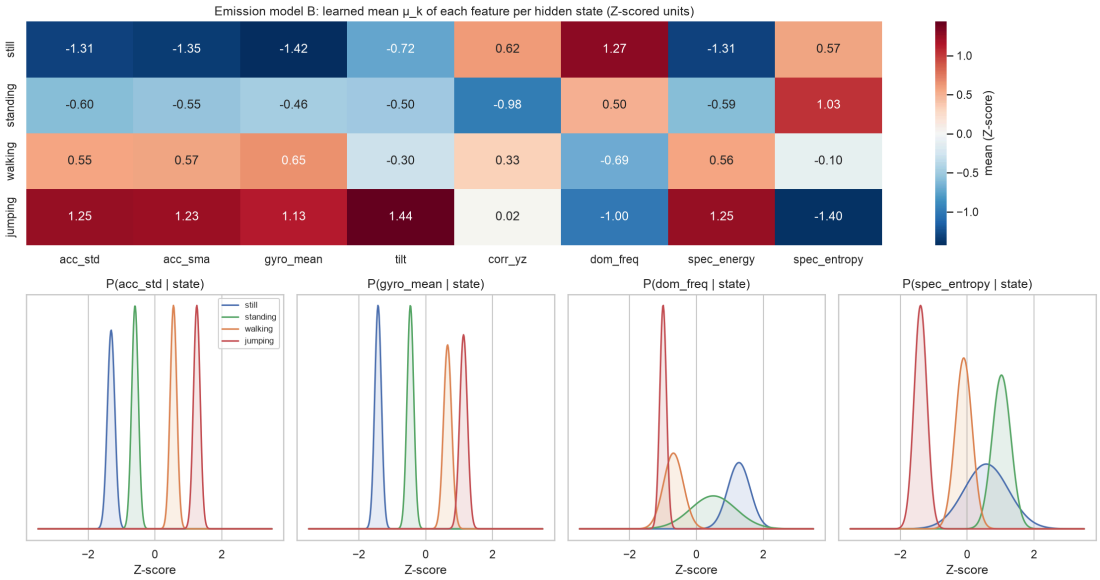


Figure 3 – Emission model: learned feature means per state (top) and Gaussian emission densities for four key features (bottom).

4. Results and interpretation

The model was evaluated on the 2 unseen living-room recordings: 80 scored windows, with 3 windows of phone handling excluded. The metrics are one-vs-rest per state. "Number of Samples" is the number of 1-s windows.

State (Activity)	Number of Samples	Sensitivity	Specificity	Overall Accuracy
Still	15	0.867	1.000	**0.862** (per-state one-vs-rest: 0.975 / 0.925 / 0.875 / 0.950)
Standing	18	0.722	0.984	
Walking	29	0.862	0.882	
Jumping	18	1.000	0.935	

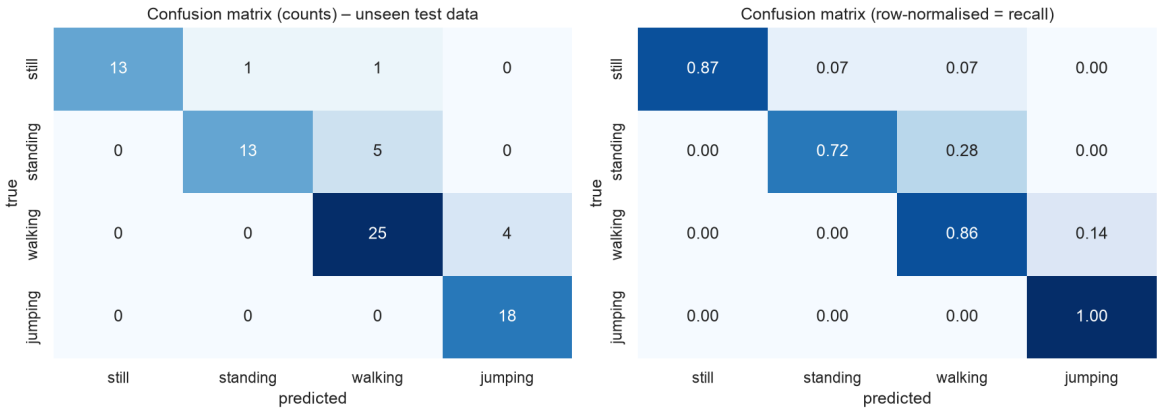


Figure 4 – Confusion matrix on unseen test data (counts and row-normalised).

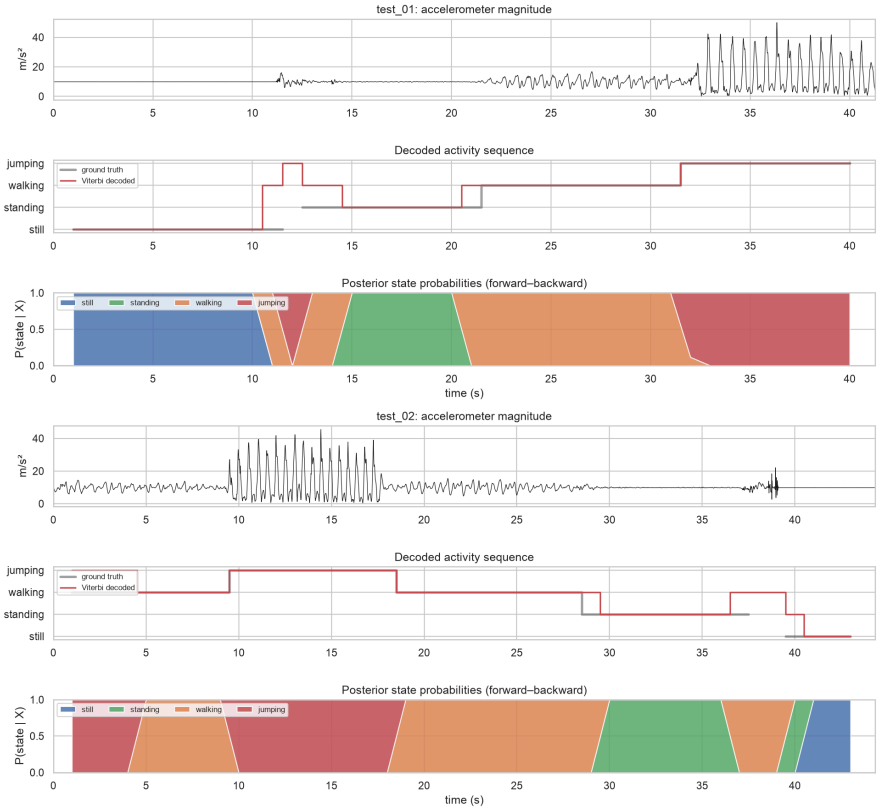


Figure 5 – Unseen recordings: signal, ground truth vs. Viterbi-decoded activity, and posterior state probabilities.

Accuracy was **89.7 %** on test_01 and **82.9 %** on test_02. Jumping was never missed (sensitivity 1.0), and still was never predicted wrongly (specificity 1.0). Most errors fall in **boundary windows**: a 2-s window that straddles a switch contains two activities. In test_01, for example, the 2 s after picking the phone up are still "settling" movement and decode as walking. The largest single error is the **first 4 s of test_02**, where walking was labelled jumping. In that segment the phone was held noticeably more tilted (gravity on the z-axis −7.7 m/s² vs. −9.6 m/s² in training), and the learned *tilt* mean is highest for jumping (Figure 3). The orientation feature therefore outweighed the intensity features.

5. Discussion and conclusion

- **Easiest vs. hardest.** *Jumping* and *still* were the easiest: their intensity and rotation features lie at opposite extremes (Figure 3), several standard deviations from the other classes. *Standing* was the hardest (sensitivity 0.72). Its signal is only small hand tremor, so whenever the user shifts the phone, it looks like the start of walking.
- **Transition probabilities.** The self-transitions of 0.87–0.93 imply average bouts of 8–14 s, matching how long each activity was actually performed. The only sizeable off-diagonal entries are physically realistic

moves: still ↔ standing (picking up or putting down the phone), standing ↔ walking, and walking ↔ jumping. Still → jumping is ≈ 0.001: nobody jumps straight from a phone lying on a table.

- **Noise and sampling rate.** At 100 Hz the signal is heavily oversampled, since human motion lies below 20 Hz, so the slight jitter (99.3 Hz) had no visible effect once the data was resampled. Sensor noise mainly affects the low-motion classes: their dominant frequency is pure noise, which gives broad, overlapping emissions (Figure 3). The very clean training clips also produced very narrow emission variances, which makes the model over-confident on unseen data. An emission-only classifier reached the same 86.2 %, meaning the emissions dominated the transition prior in this experiment.
- **Generalisation (personal reflection).** Testing in my living room instead of my bedroom cost about 14 % accuracy compared with training (100 %). Watching the decoded plots, I realised the errors come from my own behaviour more than from the model: when recording the tests I moved naturally, held the phone more tilted while walking, and took a second or two to settle after picking the phone up. My training clips were "too perfect" (one activity, same grip, same room). The model therefore generalises well to the activities themselves, but less well to transitions and to changes in how the phone is held.
- **Improvements.** (1) Collect more varied training data (different grips, pockets, rooms, and more participants), including real continuous sessions, so A is learned from genuine transitions. (2) Make features less sensitive to how the phone is held, or drop the raw *tilt* feature. (3) Use a larger variance floor or full covariances to avoid over-confident emissions. (4) Add sensors (barometer for stairs, magnetometer for heading) and a *transition* state for phone handling.

Conclusion. A Gaussian HMM implemented from scratch, with Baum–Welch training and Viterbi decoding, recognised four activities from a phone's motion sensors with **86 % accuracy on unseen recordings from a new environment**. It learned a transition structure that reflects realistic behaviour. This makes it a practical basis for the study-break monitor, which mainly needs to separate "inactive" (still/standing) from "active" (walking/jumping) reliably.